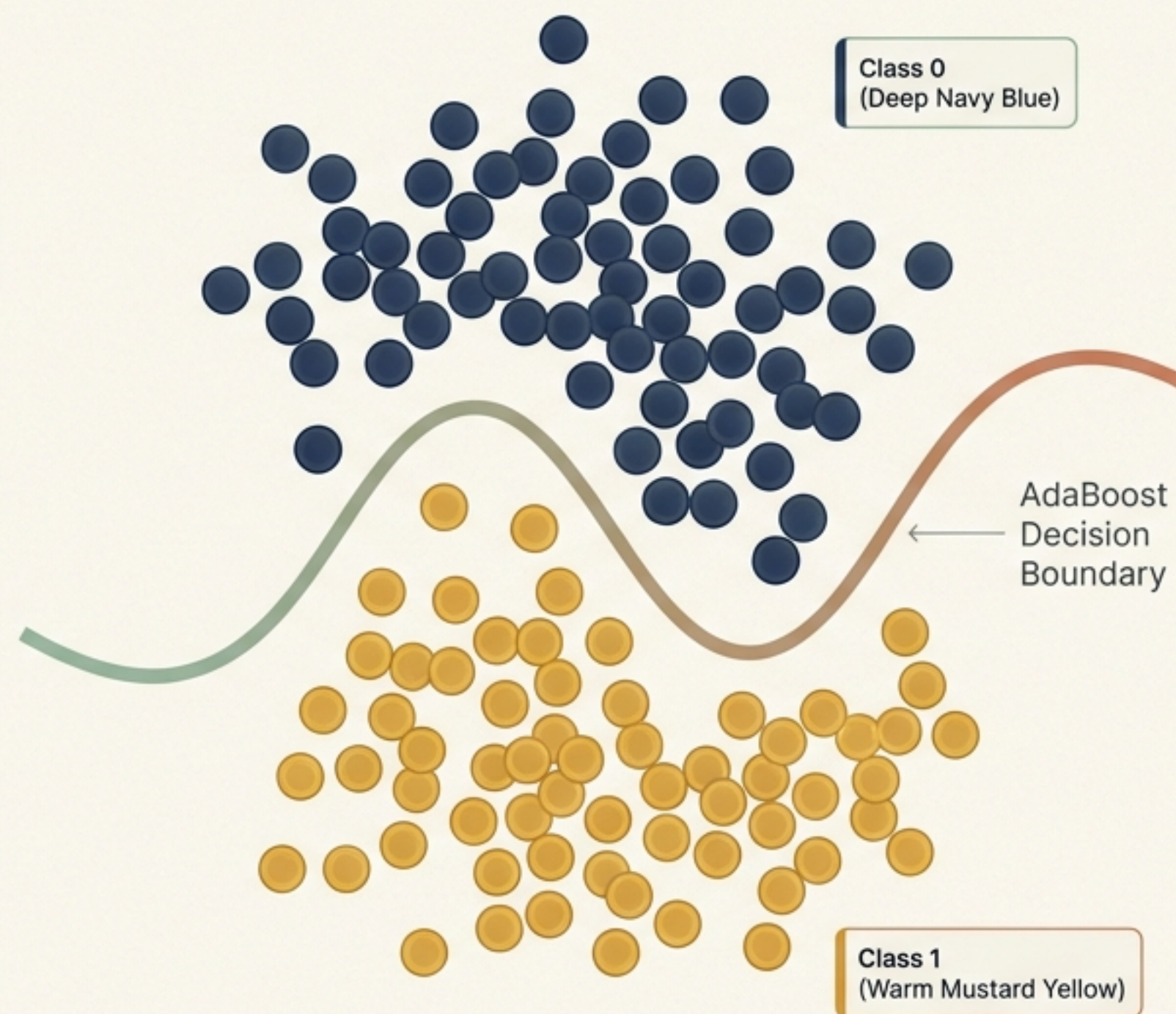


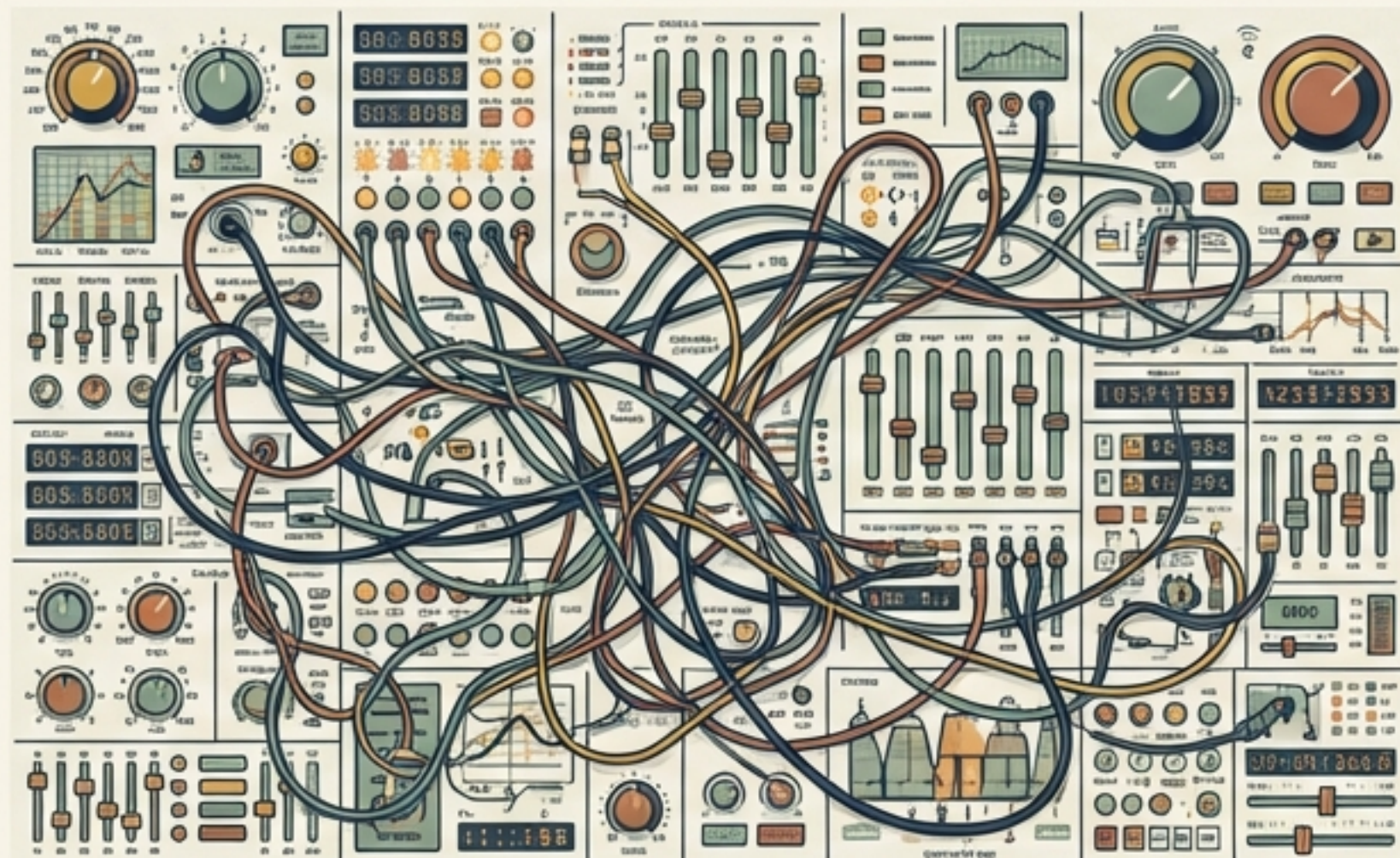
Unlocking AdaBoost

A visual guide to mastering hyperparameters, shrinkage, and model optimisation.

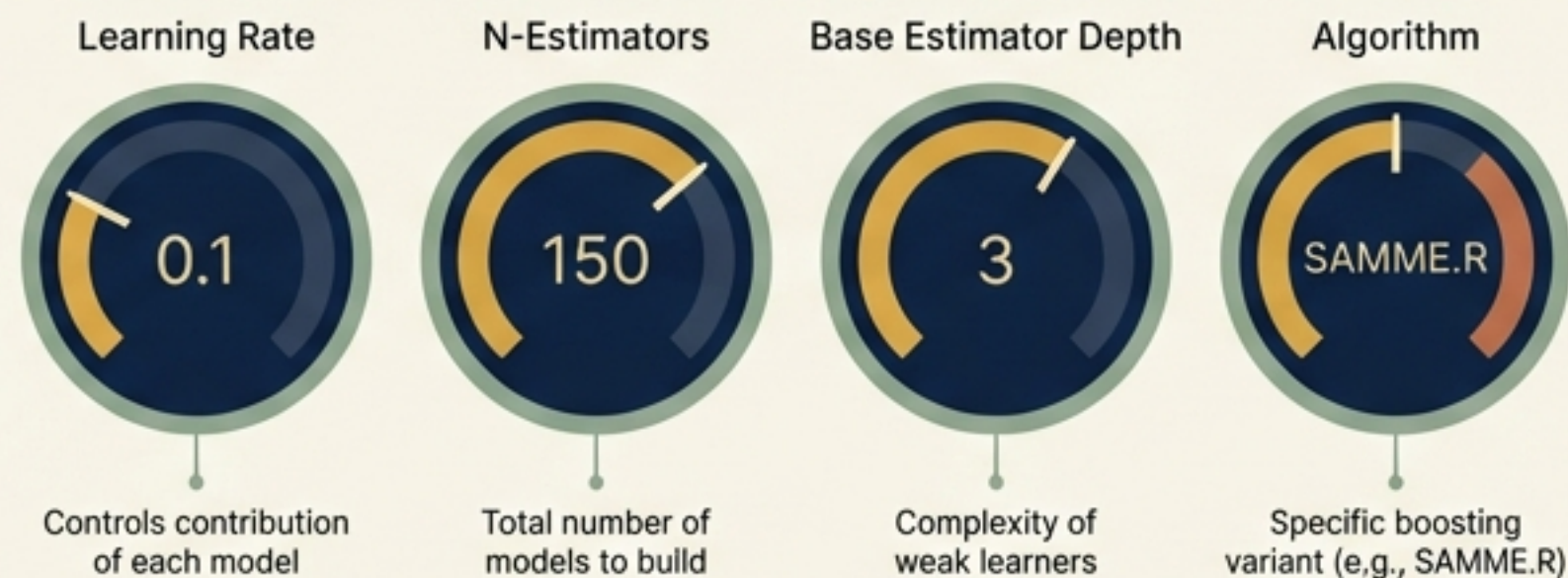


The Elegance of Simplicity

Other Boosting Algorithms



AdaBoost



Unlike XGBoost, which demands navigating a maze of hyperparameters, AdaBoost is fundamentally simpler.

It relies on precisely four core hyperparameters to dictate its predictive behaviour.

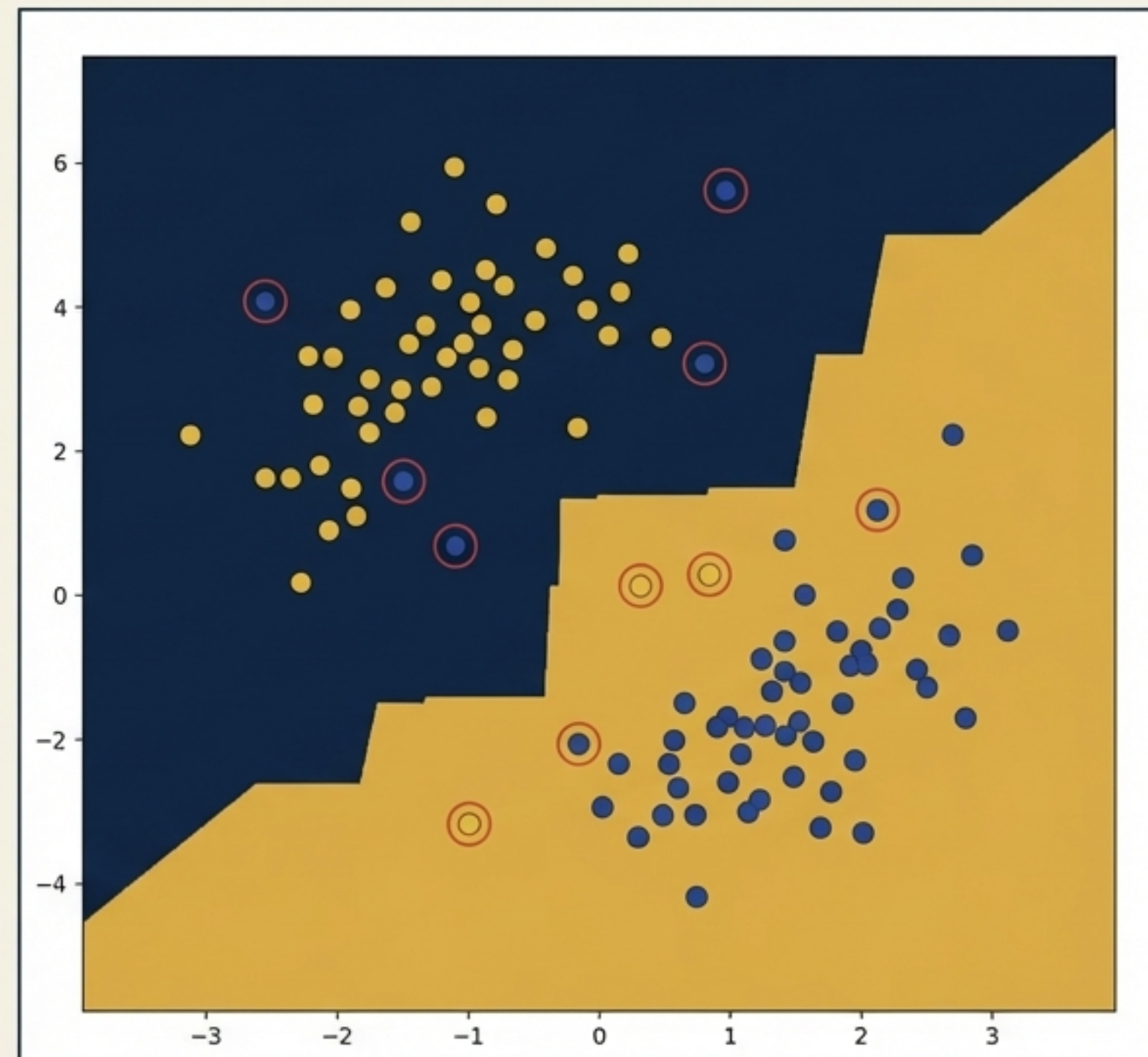
Mastering these four dials is the key to unlocking the algorithm's true power.

The Baseline Challenge: 78% Accuracy

The Setup: A custom classification dataset modelled using AdaBoost's default settings.

The Result: A baseline accuracy of 78%.

The Flaw: The model makes visible classification errors. The default parameters—while a good starting point—leave significant performance on the table.



The Four Levers of Optimisation



1. Base Estimator: The foundation.



2. Algorithm: The mathematical engine.



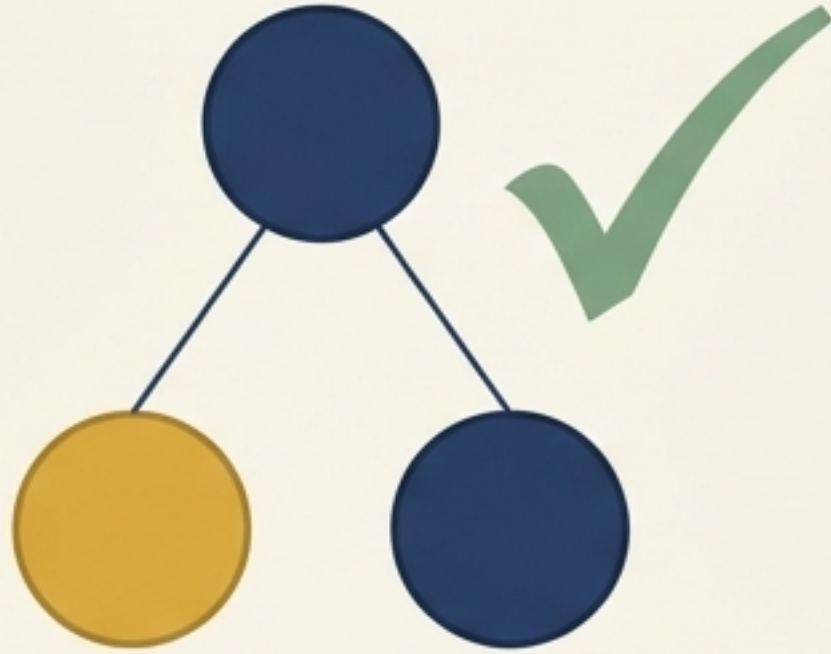
3. N-Estimators: The iteration count.



4. Learning Rate: The braking system.

While all four matter, the secret to mastering AdaBoost lies in the dynamic tension between N-Estimators and the Learning Rate.

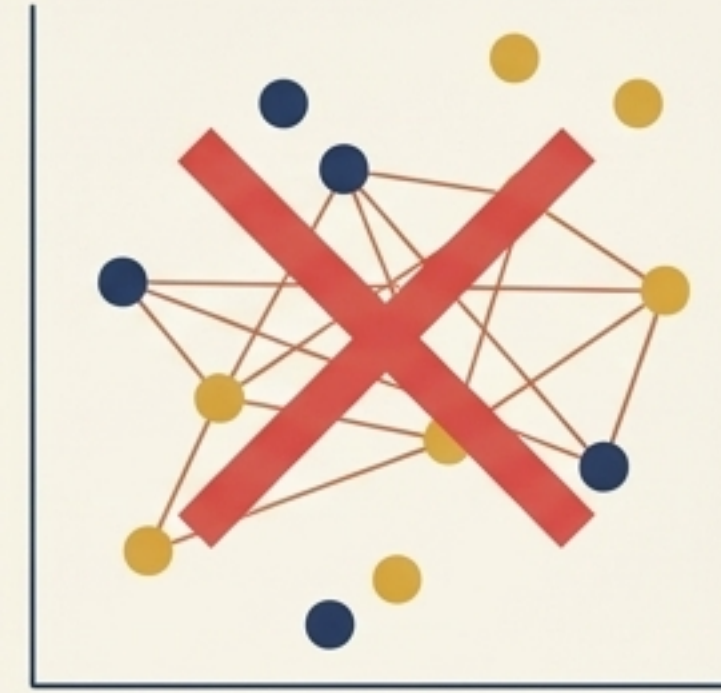
Lever 1: The Base Estimator



Decision Stump (max_depth=1)

The Golden Rule:

99.9% of the time, use a Decision Tree with max_depth=1 (a Decision Stump).

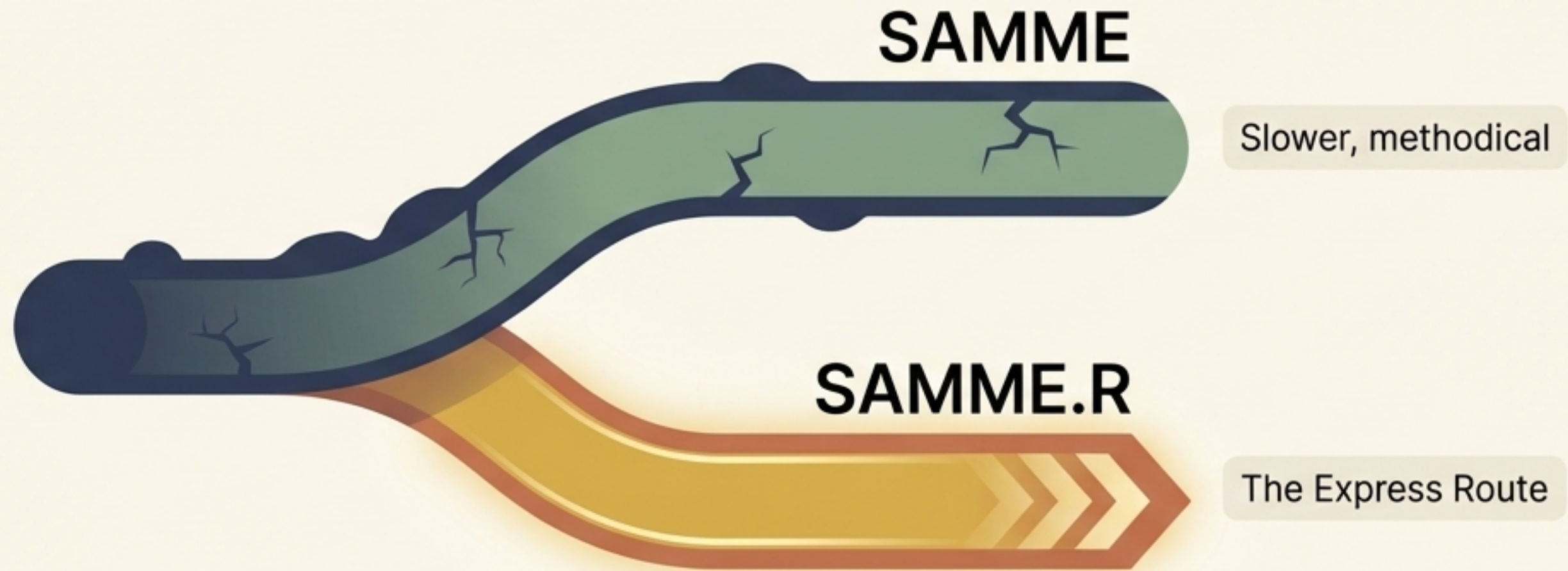


K-Nearest Neighbours

Why not others?

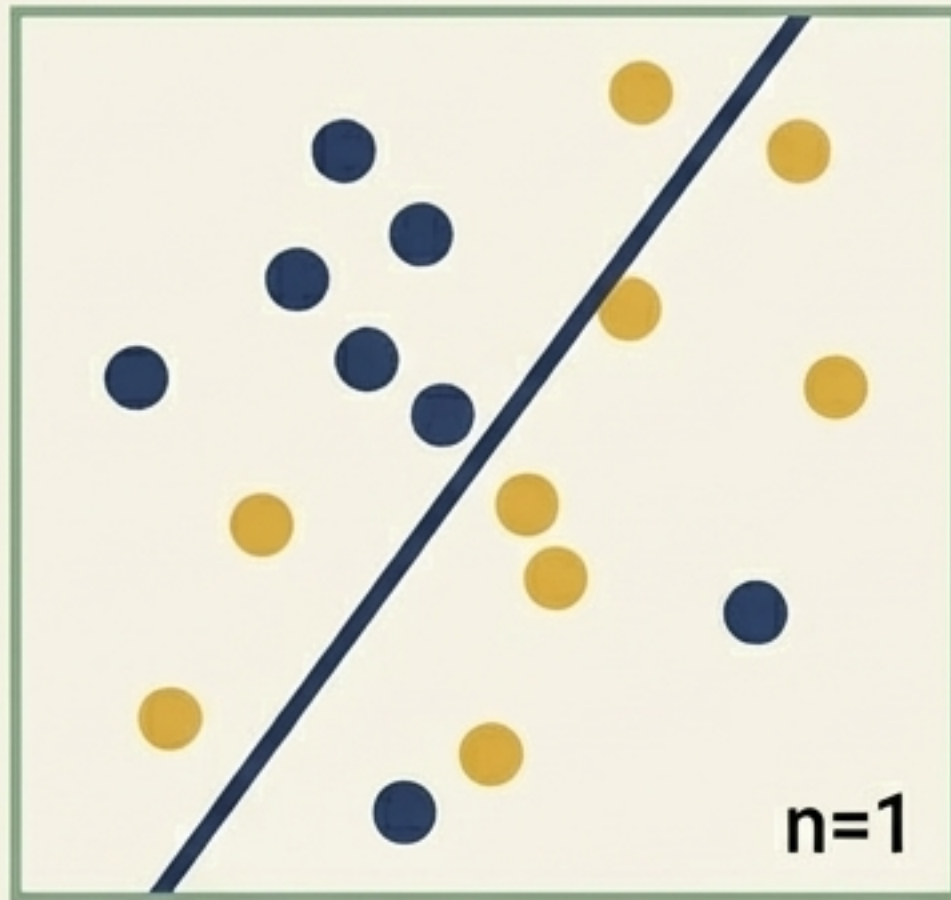
The chosen estimator must support sample weighting. Algorithms like KNN lack the feature to process sample weights inherently, rendering them useless for AdaBoost.

Lever 2: The Algorithm

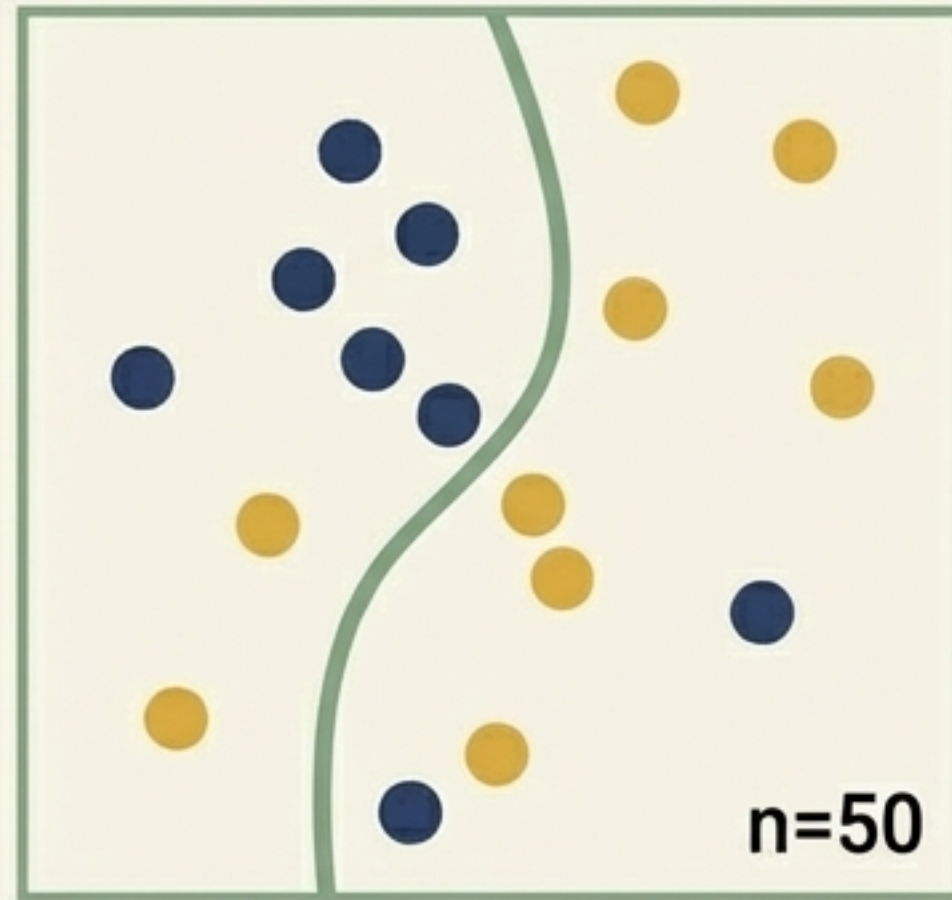


- You have two choices: SAMME or SAMME.R.
- Default to SAMME.R: It relies on real-valued probabilities rather than hard classifications.
- The Benefit: It typically converges much faster and achieves lower test errors with fewer iterations.

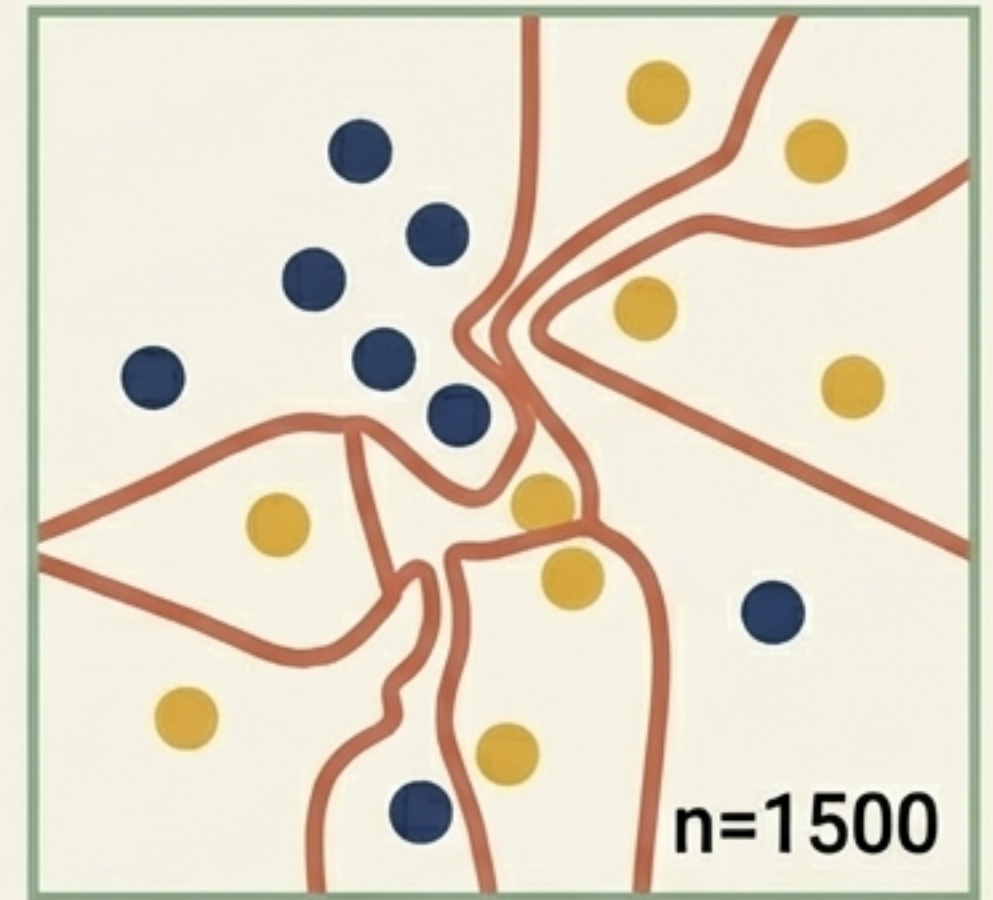
Lever 3: The Overfitting Trap ($n_estimators$)



Underfitting (Too Simplistic)



Balanced Fit



Severe Overfitting

$n_estimators$ dictates the maximum number of base models trained.

Too few leads to underfitting.

Pushing the number too high forces the model to memorise the training data, creating unstable decision regions and unnecessarily increasing training time.

Lever 4: The Braking System (`learning_rate`)

The Learning Rate multiplies directly against this value.


$$\alpha = \frac{1}{2} \log \left(\frac{1 - error}{error} \right)$$

- In AdaBoost, α determines the weight or “say” of each decision stump.
- By default, the Learning Rate is 1.0 (leaving α unchanged).
- When we reduce the Learning Rate, we artificially shrink α .

The Mechanics of Shrinkage

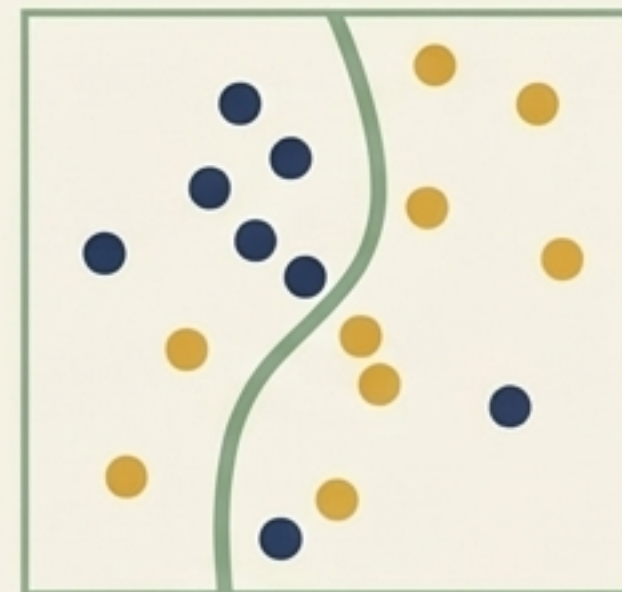
Lower Learning Rate = Smaller α

- A smaller α directly reduces the amplitude of sample weight updates at each step.
- This results in smaller variations of weighted data points, forcing the model to learn slowly.
- This slow down drastically reduces overfitting, a concept known as Shrinkage.

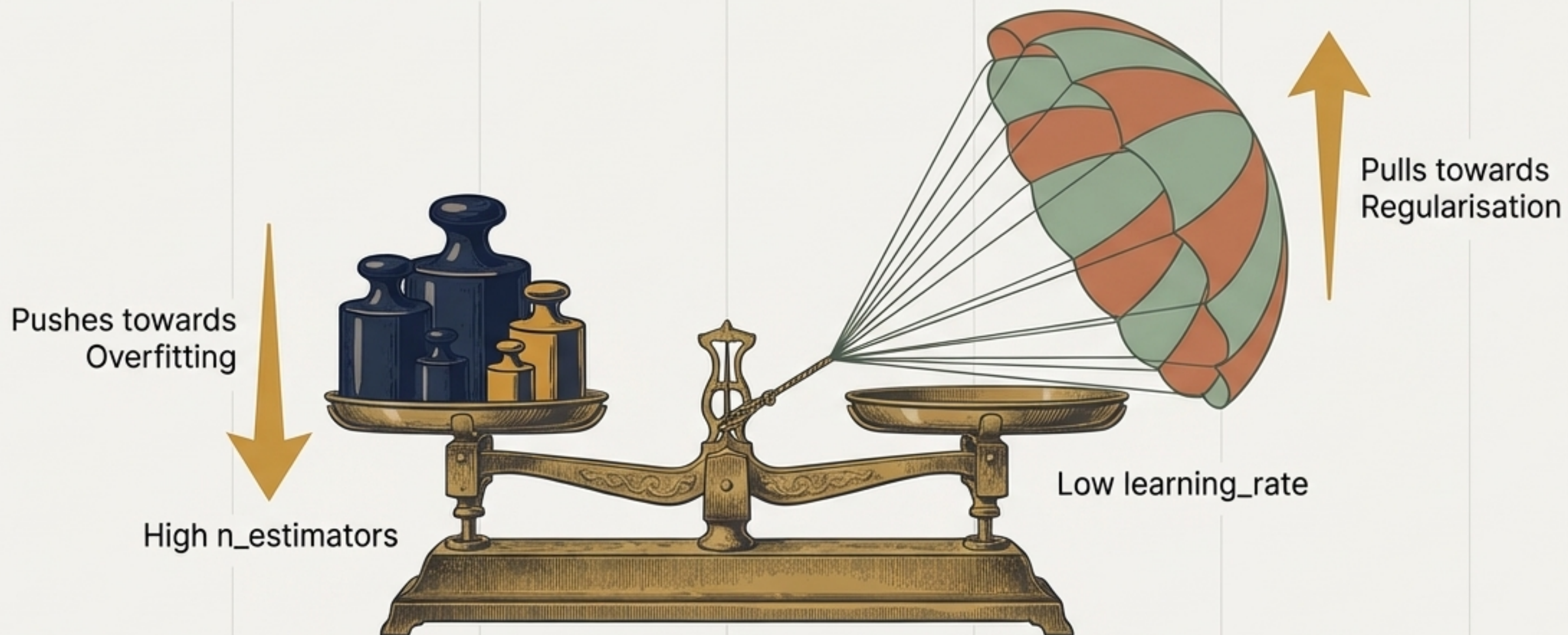
Learning Rate = 1.0



Learning Rate = 0.1



The Ultimate Tug-of-War



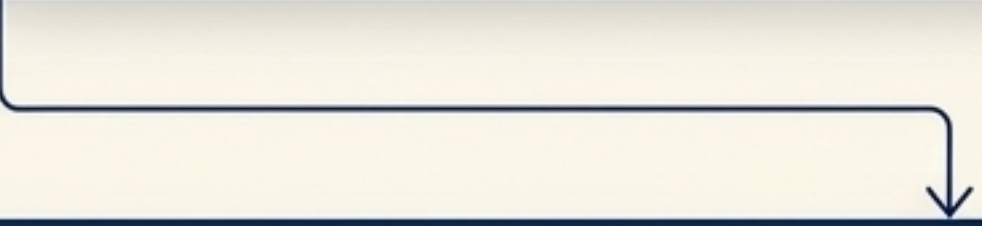
To unlock the true power of AdaBoost, these two parameters must be tuned in tandem. Increasing the number of estimators increases the risk of overfitting. You must counterbalance this by lowering the learning rate.

The Strategy: Allow the model to train for many iterations, but force it to learn very slowly during each step.

Systematising the Search (GridSearchCV)

- We translate the Tug-of-War into a systematic grid search.
- We test broad ranges of **n_estimators** against varied shrinkage penalties (**learning_rate** from 0.1 to 1.0).
- 10-fold cross-validation ensures the results are robust and not subject to random variation.

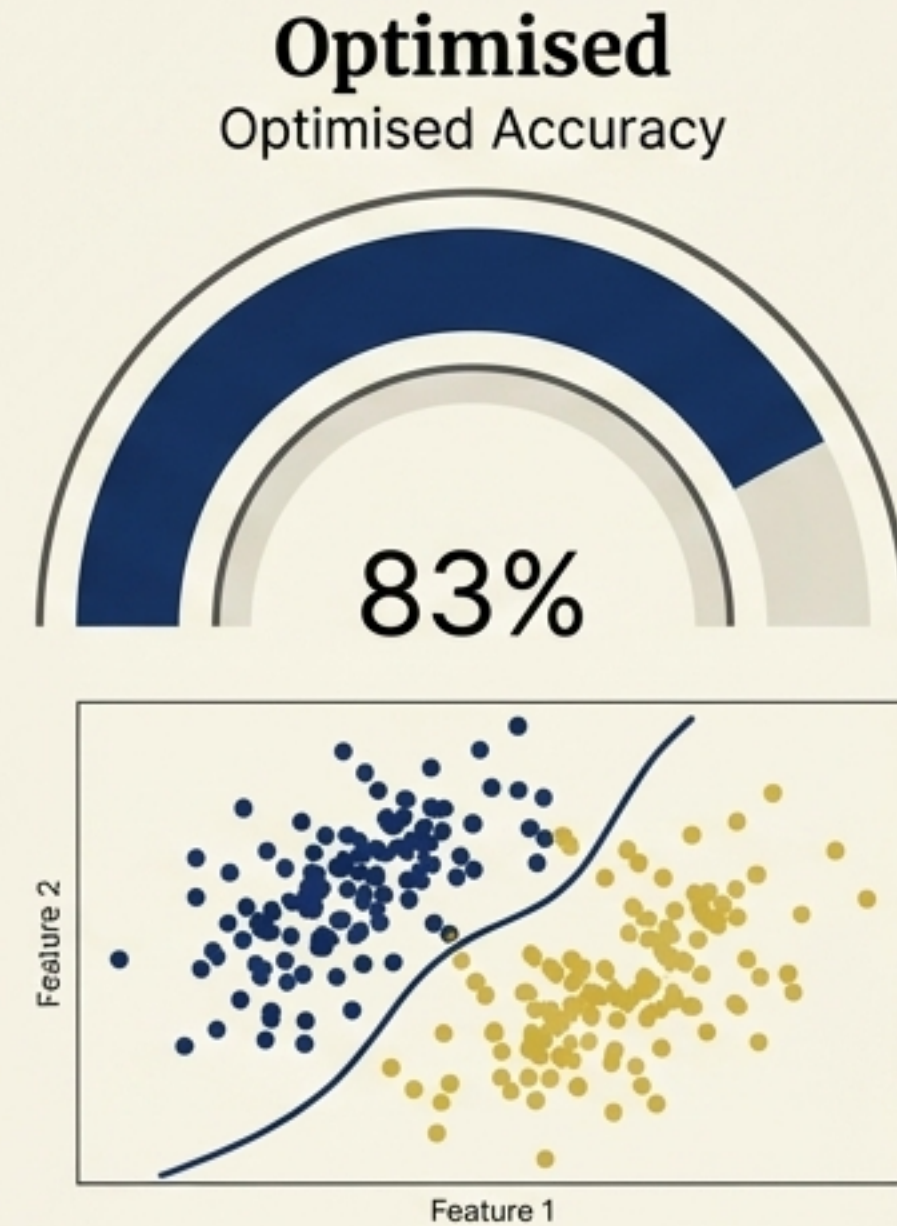
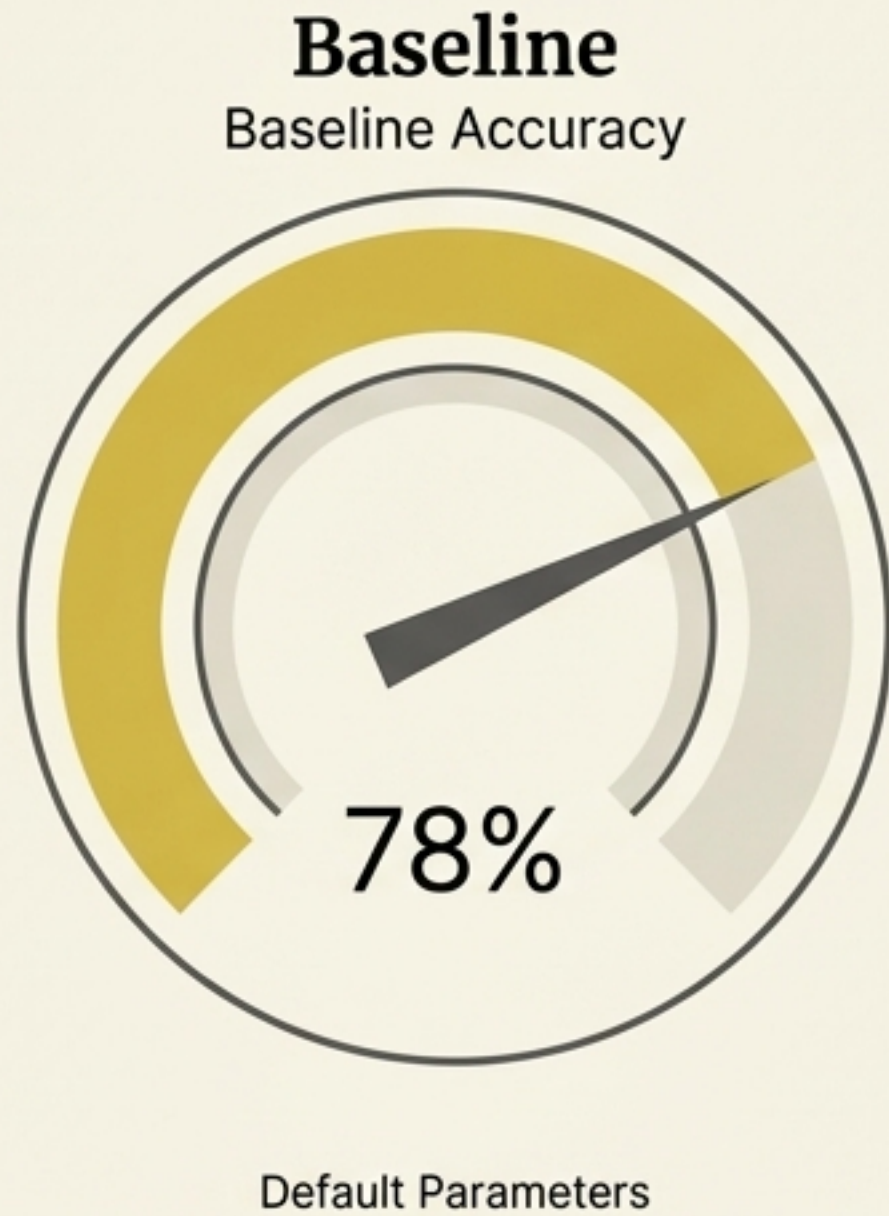
```
param_grid = {  
    'n_estimators': [50, 100, 500, ...],  
    'learning_rate': [0.1, 0.5, 1.0],  
    'algorithm': ['SAMME', 'SAMME.R']  
}
```



```
grid_search = GridSearchCV(estimator=ada_boost, param_grid=param_grid, cv=10, scoring='accuracy')
```


The Optimised Result: 83% Accuracy

Before and After



The Winning Combination

learning_rate: 0.1
(heavy shrinkage applied)

n_estimators: 500
(given plenty of room to learn)

algorithm: SAMME (surprisingly optimal for this specific dataset)

Mastering the Algorithm

Mastering the Algorithm



Embrace the Simplicity: Stick to **Decision Stumps** (`max_depth=1`) for your base estimators.



Balance the Scales: Never increase **n_estimators** without considering a reduction in your **learning_rate**.



Leverage Shrinkage: Use a low learning rate to reduce weight update amplitude and smooth out your decision boundaries.

AdaBoost is highly accessible. By mastering the interaction between iteration and learning speed, you can consistently extract its true predictive power.